Evently 開発仕様書 v2

Claude Code向け実装指示書（フェーズ2対応版）

---

0. プロジェクト概要

アプリ名：Evently **概要**：コミュニティのイベント幹事の作業負担をほぼゼロにし、参加者が事前に「誰が来るか」をプロフィール付きで確認できるイベント管理Webアプリ **フォルダ名**：`evently_app` **スタック**：Next.js 14（App Router）+ TailwindCSS + Supabase **デプロイ**：Vercel **フェーズ**：実際に使えるアプリ（テスト運用段階）

---

1. 技術スタック

```
フロントエンド：Next.js 14（App Router）+ TailwindCSS
バックエンド：Supabase（PostgreSQL + Auth + Realtime）
通知：メール（Resend）/ LINE（LINE Messaging API）
デプロイ：Vercel
```

---

2. フォルダ構成

```
evently_app/
├── app/
│   ├── page.tsx                        # トップ（幹事：イベント作成）
│   ├── dashboard/
│   │   └── page.tsx                    # 幹事：管理画面
│   ├── event/
│   │   └── [id]/
│   │       └── page.tsx                # イベントページ（役割で表示切替）
│   ├── join/
│   │   └── [id]/
│   │       └── page.tsx                # 参加申込
│   ├── join-done/
│   │   └── page.tsx                    # 参加確定
│   ├── chat/
│   │   └── [id]/
│   │       └── page.tsx                # 参加者チャット
│   └── layout.tsx
```

```
├── components/
│   ├── TestLoginBar.tsx                # テスト用アカウント切替バー
│   ├── EventForm.tsx
│   ├── ParticipantList.tsx
│   ├── JoinForm.tsx
│   ├── ChatBox.tsx
│   └── FeedbackForm.tsx               # テスト用フィードバック収集
├── lib/
│   ├── supabase.ts                     # Supabaseクライアント
│   ├── storage.ts                      # localStorageユーティリティ（開発用）
│   └── testAccounts.ts                # テストアカウント定義
└── package.json
```

---

3. データベース設計（Supabase）

テーブル一覧

```sql
```

```sql
-- コミュニティ
CREATE TABLE communities (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name          TEXT NOT NULL,
  slug          TEXT UNIQUE NOT NULL,
  admin_email   TEXT NOT NULL,
  is_private    BOOLEAN DEFAULT true,
  created_at    TIMESTAMP DEFAULT now()
);

-- メンバー（承認制・将来拡張）
CREATE TABLE members (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  community_id    UUID REFERENCES communities(id),
  auth_user_id    UUID,                        -- Supabase Auth（将来）
  name            TEXT NOT NULL,
  email           TEXT,                         -- メール通知用
  line_user_id    TEXT,                         -- LINE通知用（将来）
  notify_method   TEXT DEFAULT 'email',         -- 'line' / 'email' / 'both'
  graduation_year INTEGER,
  major           TEXT,
  company         TEXT,
  job_title       TEXT,
  bio             TEXT,
  avatar_color    TEXT,
  status          TEXT DEFAULT 'approved',      -- 今は全員approved、将来はpending/appr
  applied_at      TIMESTAMP DEFAULT now(),
  approved_at     TIMESTAMP,
  approved_by     UUID REFERENCES members(id),
  UNIQUE(community_id, email)
);

-- イベント
CREATE TABLE events (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  community_id    UUID REFERENCES communities(id),
  title           TEXT NOT NULL,
  date_start      TIMESTAMP NOT NULL,
  date_end        TIMESTAMP,
  place_public    TEXT,                         -- 参加前に見える 例：渋谷エリア
  place_private   TEXT,                         -- 参加確定後のみ
  detail          TEXT,                         -- 費用・持ち物など（参加確定後のみ）
  capacity        INTEGER,
```

```sql
  status          TEXT DEFAULT 'open',          -- open / closed / cancelled
  created_at      TIMESTAMP DEFAULT now()
);

-- イベントメンバー（幹事 or 参加者・1人が両方になれる・複数幹事対応）
CREATE TABLE event_members (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id    UUID REFERENCES events(id),
  member_id   UUID REFERENCES members(id),
  role        TEXT NOT NULL,                    -- 'organizer'（幹事）/ 'participant'
  joined_at   TIMESTAMP DEFAULT now(),
  UNIQUE(event_id, member_id, role)
);

-- チャット
CREATE TABLE chat_messages (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id    UUID REFERENCES events(id),
  member_id   UUID REFERENCES members(id),
  body        TEXT NOT NULL,
  sent_at     TIMESTAMP DEFAULT now()
);

-- LINEグループ（将来拡張）
CREATE TABLE line_groups (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id        UUID REFERENCES events(id),
  line_group_id   TEXT,
  invite_link     TEXT,
  created_at      TIMESTAMP DEFAULT now()
);

-- リマインド管理
CREATE TABLE reminders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id    UUID REFERENCES events(id),
  remind_at   TIMESTAMP NOT NULL,
  channel     TEXT NOT NULL,                    -- 'line' / 'email'
  status      TEXT DEFAULT 'pending',           -- pending / sent / failed
  sent_at     TIMESTAMP,
  created_at  TIMESTAMP DEFAULT now()
);

-- フィードバック（テスト収集用）
```

```sql
CREATE TABLE feedbacks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id    UUID REFERENCES events(id),
  member_id   UUID REFERENCES members(id),
  rating      TEXT,                           -- 'good' / 'okay' / 'bad'
  comment     TEXT,
  created_at  TIMESTAMP DEFAULT now()
);
```

---

4. テストアカウント

`lib/testAccounts.ts` に以下を定義すること。

```typescript
```

```
export type TestAccount = {
  id: string
  name: string
  role: 'organizer' | 'participant'
  email: string
  graduation_year: number
  major: string
  company: string
  job_title: string
  bio: string
  avatar_color: string
}

export const TEST_ACCOUNTS: TestAccount[] = [
  // 幹事（2名）
  {
    id: 'test-organizer-1',
    name: '山本 幹事',
    role: 'organizer',
    email: 'organizer1@test.com',
    graduation_year: 2018,
    major: '経営戦略',
    company: '〇〇コンサルティング',
    job_title: 'シニアマネージャー',
    bio: 'イベント企画が得意です',
    avatar_color: '#E8F5E9',
  },
  {
    id: 'test-organizer-2',
    name: '佐々木 副幹事',
    role: 'organizer',
    email: 'organizer2@test.com',
    graduation_year: 2019,
    major: 'マーケティング',
    company: '〇〇広告代理店',
    job_title: 'マネージャー',
    bio: '一緒に盛り上げましょう！',
    avatar_color: '#E3F2FD',
  },
  // 参加者（10名）
  {
    id: 'test-participant-1',
    name: '田中 美咲',
```

```
    role: 'participant',
    email: 'p1@test.com',
    graduation_year: 2020,
    major: 'マーケティング',
    company: 'スタートアップ A',
    job_title: 'CMO',
    bio: 'マーケ・ブランド戦略が専門です',
    avatar_color: '#FCE4EC',
  },
  {
    id: 'test-participant-2',
    name: '鈴木 健太',
    role: 'participant',
    email: 'p2@test.com',
    graduation_year: 2019,
    major: 'ファイナンス',
    company: '外資系 PE ファンド',
    job_title: 'ディレクター',
    bio: 'M&A・投資案件を担当しています',
    avatar_color: '#FFF3E0',
  },
  {
    id: 'test-participant-3',
    name: '佐藤 由美',
    role: 'participant',
    email: 'p3@test.com',
    graduation_year: 2021,
    major: '起業家精神',
    company: '自社（SaaS）',
    job_title: '代表取締役',
    bio: 'B2B SaaSを創業しました',
    avatar_color: '#F3E5F5',
  },
  {
    id: 'test-participant-4',
    name: '伊藤 大輔',
    role: 'participant',
    email: 'p4@test.com',
    graduation_year: 2018,
    major: 'オペレーション',
    company: '大手製造業',
    job_title: '事業部長',
    bio: 'サプライチェーン改革に取り組んでいます',
    avatar_color: '#E8F5E9',
```

```
  },
  {
    id: 'test-participant-5',
    name: '渡辺 さくら',
    role: 'participant',
    email: 'p5@test.com',
    graduation_year: 2022,
    major: 'イノベーション',
    company: '外資系テック企業',
    job_title: 'プロダクトマネージャー',
    bio: 'プロダクト開発・UXが得意です',
    avatar_color: '#E3F2FD',
  },
  {
    id: 'test-participant-6',
    name: '中村 隆',
    role: 'participant',
    email: 'p6@test.com',
    graduation_year: 2017,
    major: 'グローバルリーダーシップ',
    company: '総合商社',
    job_title: '部長',
    bio: 'アジア事業を統括しています',
    avatar_color: '#FFF9C4',
  },
  {
    id: 'test-participant-7',
    name: '小林 恵',
    role: 'participant',
    email: 'p7@test.com',
    graduation_year: 2020,
    major: 'ヘルスケア経営',
    company: '医療スタートアップ',
    job_title: 'COO',
    bio: 'ヘルステック領域で事業拡大中です',
    avatar_color: '#FCE4EC',
  },
  {
    id: 'test-participant-8',
    name: '加藤 誠',
    role: 'participant',
    email: 'p8@test.com',
    graduation_year: 2019,
    major: 'テクノロジー経営',
```

```
    company: 'VC ファンド',
    job_title: 'アソシエイト',
    bio: 'スタートアップ投資が専門です',
    avatar_color: '#F3E5F5',
  },
  {
    id: 'test-participant-9',
    name: '山田 花子',
    role: 'participant',
    email: 'p9@test.com',
    graduation_year: 2021,
    major: 'サステナビリティ',
    company: '外資系コンサル',
    job_title: 'コンサルタント',
    bio: 'ESG・サステナビリティ戦略を担当',
    avatar_color: '#E8F5E9',
  },
  {
    id: 'test-participant-10',
    name: '松本 拓也',
    role: 'participant',
    email: 'p10@test.com',
    graduation_year: 2018,
    major: 'アントレプレナーシップ',
    company: '自社（EC）',
    job_title: '代表取締役',
    bio: 'EC事業を経営しています。M&A検討中',
    avatar_color: '#FFF3E0',
  },
]

export const TEST_ORGANIZERS = TEST_ACCOUNTS.filter(a => a.role === 'organizer')
export const TEST_PARTICIPANTS = TEST_ACCOUNTS.filter(a => a.role === 'participant')
```

---

5. テスト用ログインバー

`components/TestLoginBar.tsx` を作成し、**開発・ステージング環境でのみ表示**すること。

```
┌──────────────────────────────────────────┐
│ テストモード　ログイン切替：                │
│ [山本 幹事] [佐々木 副幹事]                 │
│ [田中] [鈴木] [佐藤] [伊藤] [渡辺]          │
│ [中村] [小林] [加藤] [山田] [松本]          │
└──────────────────────────────────────────┘
```

実装仕様：

- `process.env.NEXT_PUBLIC_ENV === 'development'` のときのみ表示
- 選択したアカウントを `sessionStorage` に保存
- 画面上部に固定表示（sticky）
- 現在ログイン中のアカウントをハイライト表示

---

6. 役割による画面の出し分け

`app/event/[id]/page.tsx` で以下のロジックを実装すること。

```typescript
// sessionStorageから現在のテストアカウントを取得
const currentUser = getTestAccount() // lib/testAccounts.ts から

// event_membersテーブルで役割を確認
const myRole = await getMyRole(eventId, currentUser.id)
// 'organizer' / 'participant' / null（未参加）

// 役割によって表示を切り替え
if (myRole === 'organizer') {
  // 幹事UI：参加者管理・イベント編集ボタン表示
} else if (myRole === 'participant') {
  // 参加者UI：詳細情報・チャットボタン表示
} else {
  // 未参加UI：参加するボタン表示
}
```

---

7. 画面仕様

画面①：イベント作成（`/`）

入力項目：

- イベント名（必須）
- コミュニティ名（必須）
- 日時（必須）
- 公開用の場所（必須）例：渋谷エリア
- 実際の場所・住所（必須・参加確定者のみ表示）
- 定員（必須）
- 参加確定者のみに送る詳細情報（任意）

作成ボタン押下後：

- Supabaseの`events`テーブルに保存
- `event_members`に幹事として登録
- `/dashboard`へ遷移

画面②：幹事管理画面（`/dashboard`）

表示内容：

- イベントページのURL（コピーボタン付き）
- 「このURLをLINEグループに貼ってください」案内
- 参加者数 / 定員・残り席数
- 参加者リスト（名前・申込時刻）
- リマインド設定（前日・3日前・1週間前から選択）

画面③：イベントページ（`/event/[id]`）

全員に見える：

- イベント名・日時・公開用の場所
- 参加者数 / 定員
- 参加者一覧（名前・卒業年度・現職・アバター）

幹事のみ見える：

- 参加者管理ボタン
- イベント編集ボタン
- 実際の場所・詳細情報

参加者のみ見える：

- 実際の場所・詳細情報
- チャットボタン

未参加のみ見える：

- 「参加する」ボタン

画面④：参加申込（`/join/[id]`）

入力項目：

- お名前（必須）
- 卒業年度・専攻（必須）
- 現職（任意）
- メールアドレス（必須）

申込後：

- `event_members` に `role: 'participant'` で保存
- `/join-done` へ遷移

画面⑤：参加確定（`/join-done`）

表示内容：

- 参加確定メッセージ
- 詳細情報（場所・費用・持ち物）
- メールプレビュー（モック）
- フィードバックフォーム（テスト用）
- 「参加者一覧に戻る」ボタン

- 「チャットを見る」ボタン

画面⑥：チャット（`/chat/[id]`）

- 参加者チャット（event_membersのparticipant + organizerのみ閲覧・投稿可）
- Supabase Realtimeでリアルタイム更新
- メッセージはSupabaseの `chat_messages` テーブルに保存

画面⑦：フィードバック収集（参加確定後に表示）

```
使ってみていかがでしたか？
[とても使いやすかった] [まあまあ] [使いにくかった]

一言コメント：
[                          ]
[送信する]
```

回答は `feedbacks` テーブルに保存。

---

8. 通知設計

メール通知（Resend使用）

```
① 参加確定メール
  タイミング：参加申込直後
  宛先：参加者のメールアドレス
  内容：詳細情報・イベントページURL

② 前日リマインドメール
  タイミング：イベント前日の朝9時（Supabase cronで自動送信）
  宛先：全参加者
  内容：日時・場所・参加者数・イベントページURL
```

LINE通知（将来拡張・今回はモック表示のみ）

```
line_groups テーブルに invite_link を登録しておく
参加確定後の画面にLINEグループの招待リンクを表示するだけ
```

```
（実際のLINE Messaging API連携は次フェーズ）
```

---

9. デモ用初期データ

アプリ起動時・Supabaseが空の場合に自動生成すること。

```typescript
// デモコミュニティ
const demoCommunity = {
  name: 'MBA同窓会',
  slug: 'mba-alumni',
  admin_email: 'admin@test.com',
}

// デモイベント
const demoEvent = {
  title: 'MBA同窓生 春の交流会2026',
  date_start: '2026-04-15T19:00:00+09:00',
  date_end: '2026-04-15T21:00:00+09:00',
  place_public: '渋谷エリア',
  place_private: '渋谷区道玄坂1-2-3 ビル5F',
  detail: '📍 渋谷区道玄坂1-2-3 ビル5F｜💰 4,000円（当日精算）｜👔 平服でお越しください',
  capacity: 20,
}

// デモ参加者：TEST_ACCOUNTSの参加者10名全員を登録
// TEST_ORGANIZERSの2名を幹事として登録
```

---

10. 環境変数（.env.local）

```env
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
NEXT_PUBLIC_ENV=development     # 本番では production に変更
RESEND_API_KEY=your_resend_api_key
```

---

11. 起動・デプロイ手順（README.mdに記載すること）

```bash
# ローカル起動
cd evently_app
npm install
npm run dev
# → http://localhost:3000

# Vercelデプロイ
# 1. GitHubにpush
# 2. vercel.com でリポジトリ連携
# 3. 環境変数を設定
# 4. 自動デプロイ完了
```

---

12. テスト手順

1人でテストする場合（動作確認）

```
1. ブラウザを3つ開く（Chrome・Safari・シークレット）
2. それぞれ別のテストアカウントに切り替える
3. シナリオ通りに操作する
```

3人でテストする場合（UX確認）

```
シナリオ：
1. あなた（山本 幹事）：
   イベントを作成 → URLをLINEグループに送る

2. テスター①（田中 美咲）：
   URLを開く → 参加者一覧を確認 → 参加申込

3. テスター②（鈴木 健太）：
   URLを開く → 田中さんが表示されているか確認 → 参加申込

4. あなた（幹事に戻る）：
   管理画面で2名が増えているか確認
```

テスト環境URL

```
Vercelのプレビュー機能を使う：
feature ブランチにpush → 自動でプレビューURLが生成される
→ そのURLを3人に送る
```

---

13. フェーズロードマップ

```
【フェーズ1（完了）】
localStorageのデモ用プロトタイプ

【フェーズ2（今回）】
Supabase + メール通知 + テストアカウント
実際に3人で使えるアプリ

【フェーズ3（次）】
LINE Messaging API連携
LINEグループ自動招待
Supabase Authでログイン実装

【フェーズ4（将来）】
承認制メンバーデータベース
members.status = 'pending' / 'approved'
コミュニティ管理者画面
```

---

14. スコープ外（今回実装しない）

- Supabase Authによる本格的なログイン（テストアカウント切替で代替）
- LINE Messaging API実連携（モック表示のみ）
- 承認制メンバーフロー（全員approved固定）
- 決済機能